Aritmética de precisión arbitraria

La aritmética de precisión arbitraria, también conocida como "bignum" o simplemente "aritmética larga", es un conjunto de estructuras de datos y algoritmos que permiten procesar números mucho mayores que los que caben en los tipos de datos estándar. A continuación se presentan varios tipos de aritmética de precisión arbitraria.

Aritmética larga de enteros clásicos

La idea principal es que el número se almacena como una matriz de sus "dígitos" en una base. Algunas de las bases más utilizadas son la decimal y las potencias decimales (10^4 o 10^9) y binario.
Las operaciones con números en esta forma se realizan mediante algoritmos tradicionales de suma, resta, multiplicación y división en columnas. También es posible utilizar algoritmos de multiplicación rápida: la transformada rápida de Fourier y el algoritmo de Karatsuba.
Aquí describimos operaciones aritméticas largas solo para enteros no negativos. Para extender los algoritmos y que gestionen enteros negativos, es necesario introducir y mantener una bandera adicional de "número negativo" o usar la representación de enteros en complemento a dos.

Estructura de datos

Almacenaremos los números como un vector<int>, en el que cada elemento es un solo "dígito" del número.

```
typedef vector<int> lnum;
```

Para mejorar el rendimiento utilizaremos 10^9 como base, de modo que cada "dígito" del número largo contenga 9 dígitos decimales a la vez.

```
const int base = 1000*1000*1000;
```

Los dígitos se almacenarán en orden de menor a mayor. Todas las operaciones se implementarán de forma que, después de cada una, el resultado no tenga ceros a la izquierda, siempre que los operandos tampoco los tengan. Todas las operaciones que puedan generar un número con ceros a la izquierda deben ir seguidas de un código que los elimine. Tenga en cuenta que en esta representación existen dos notaciones válidas para el número cero: un vector vacío y un vector con un solo dígito cero.

Producción

Imprimir el entero largo es la operación más sencilla. Primero imprimimos el último elemento del vector (o 0 si el vector está vacío), seguido del resto de los elementos, rellenados con ceros iniciales si es necesario, para que tengan exactamente 9 dígitos.

```
printf ("%d", a.empty() ? 0 : a.back());
for (int i=(int)a.size()-2; i>=0; --i)
    printf ("%09d", a[i]);
```

Tenga en cuenta que convertimos a.size() a entero para evitar un desbordamiento de entero sin signo si el vector contiene menos de 2 elementos.

Aporte

Para leer un entero largo, lea su notación en una string y luego conviértalo a "dígitos":

```
for (int i=(int)s.length(); i>0; i-=9)
    if (i < 9)
        a.push_back (atoi (s.substr (0, i).c_str()));
    else
        a.push_back (atoi (s.substr (i-9, 9).c_str()));
```

Si usamos una matriz de char en lugar de una string, el código será aún más corto:

```
for (int i=(int)strlen(s); i>0; i-=9) {
    s[i] = 0;
    a.push_back (atoi (i>=9 ? s+i-9 : s));
}
```

Si la entrada puede contener ceros iniciales, se pueden eliminar de la siguiente manera:

```
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Suma

Incrementar entero largo a por b y almacenar el resultado en a:

```
int carry = 0;
for (size_t i=0; i<max(a.size(),b.size()) || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    a[i] += carry + (i < b.size() ? b[i] : 0);
    carry = a[i] >= base;
    if (carry)  a[i] -= base;
}
```

Sustracción

Decrementar entero largo a por b (a ≥ b) y almacenar el resultado en a:

```
int carry = 0;
for (size_t i=0; i<b.size() || carry; ++i) {
    a[i] -= carry + (i < b.size() ? b[i] : 0);
    carry = a[i] < 0;
    if (carry)  a[i] += base;
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Tenga en cuenta que después de realizar la resta, eliminamos los ceros iniciales para cumplir con la premisa de que nuestros números enteros largos no tienen ceros iniciales.

Multiplicación por entero corto

Multiplicar un entero largo a por entero corto b (b < base) y almacenar el resultado en a:

```
int carry = 0;
for (size_t i=0; i<a.size() || carry; ++i) {
    if (i == a.size())
        a.push_back (0);
    long long cur = carry + a[i] * 1ll * b;
    a[i] = int (cur % base);
    carry = int (cur / base);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Optimización adicional: Si el tiempo de ejecución es extremadamente importante, puede intentar reemplazar dos divisiones por una encontrando solo el resultado entero de la división (variable carry) y luego usarlo para hallar el módulo mediante la multiplicación. Esto suele acelerar el código, aunque no de forma drástica.

Multiplicación por entero largo

Multiplicar números enteros largos a y b y almacenar el resultado en c:

```
lnum c (a.size()+b.size());
for (size_t i=0; i<a.size(); ++i)
   for (int j=0, carry=0; j<(int)b.size() || carry; ++j) {
      long long cur = c[i+j] + a[i] * 1ll * (j < (int)b.size() ? b[j] : 0) + carry;
      c[i+j] = int (cur % base);
```

```
      carry = int (cur / base);
   }
while (c.size() > 1 && c.back() == 0)
   c.pop_back();
```

División por entero corto

Dividir entero largo a por entero corto b (b < base), almacena el resultado entero en a y resto en carry:

```
int carry = 0;
for (int i=(int)a.size()-1; i>=0; --i) {
    long long cur = a[i] + carry * 1ll * base;
    a[i] = int (cur / b);
    carry = int (cur % b);
}
while (a.size() > 1 && a.back() == 0)
    a.pop_back();
```

Aritmética de enteros largos para la representación de factorización

La idea es almacenar el número entero como su factorización, es decir, las potencias de los primos que lo dividen.

Este enfoque es muy fácil de implementar y permite realizar multiplicaciones y divisiones fácilmente (asintóticamente más rápido que el método clásico), pero no sumas ni restas. Además, consume mucha menos memoria que el método clásico.

Este método se utiliza a menudo para cálculos módulo número no primo M; en este caso, un número se almacena como potencias de divisores de M que dividen el número, más el resto módulo M.

Aritmética de enteros largos en módulos primos (Algoritmo de Garner)

La idea es elegir un conjunto de números primos (normalmente son lo suficientemente pequeños como para caber en el tipo de datos entero estándar) y almacenar un entero como un vector de residuos de la división del entero por cada uno de esos primos.

El teorema del resto chino establece que esta representación es suficiente para restaurar de forma única cualquier número desde 0 al producto de estos primos menos uno. El algoritmo de Garner permite restaurar el número de dicha representación a un entero normal.

Este método permite ahorrar memoria en comparación con el enfoque clásico (aunque el ahorro no es tan significativo como en la representación factorizada). Además, permite realizar sumas, restas y multiplicaciones rápidas en un tiempo proporcional al número de números primos utilizados como módulo (véase el artículo sobre el teorema del resto chino para su implementación).

La desventaja es que convertir el entero a su forma normal es bastante laborioso y requiere implementar aritmética clásica de precisión arbitraria con multiplicación. Además, este método no admite la división.

Aritmética de precisión arbitraria fraccionaria

Las fracciones aparecen en las competiciones de programación con menos frecuencia que los números enteros, y la aritmética larga es mucho más complicada de implementar para las fracciones, por lo que las competiciones de programación presentan solo un pequeño subconjunto de aritmética larga fraccionaria.

Aritmética en fracciones irreducibles

Un número se representa como una fracción irreducible a/b, dónde a y b son enteros. Todas las operaciones con fracciones pueden representarse como operaciones con los numeradores y denominadores enteros de dichas fracciones. Normalmente, esto requiere usar aritmética clásica de

precisión arbitraria para almacenar numerador y denominador, pero a veces basta con un tipo de dato entero integrado de 64 bits.

Almacenamiento de la posición de punto flotante como tipo separado

A veces, un problema requiere manejar números muy pequeños o muy grandes sin permitir desbordamientos ni subdesbordamientos. El tipo de dato doble integrado usa de 8 a 10 bytes y permite valores del exponente en [-308; 308] alcance, que a veces puede ser insuficiente.

El enfoque es muy simple: se utiliza una variable entera separada para almacenar el valor del exponente y, después de cada operación, el número de punto flotante se normaliza, es decir, se devuelve a [0.1; 1) intervalo ajustando el exponente en consecuencia.

Al multiplicar o dividir dos números, se deben sumar o restar sus exponentes, respectivamente. Para sumar o restar números, primero deben elevarse a exponente común multiplicando uno de ellos por 10 elevado a la potencia igual a la diferencia de los valores de los exponentes.

Como nota final, la base del exponente no tiene que ser necesariamente igual a 10. Según la representación interna de los números de punto flotante, lo más lógico es utilizar 2 como base del exponente.